PF

LAB#07

NAME: ALIZA SHAMSHER

ROLL NO: 26K-2548

Q#01:

[*] comb.c | Untitled2.cpp | combination.c | Untitled4.cpp

```c
#include <stdio.h>

int main() {
    float mat[3][3], trans[3][3], cof[3][3], adj[3][3], inv[3][3];
    int i, j;

    printf("enter elements of 3x3 matrix:\n");
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            scanf("%f", &mat[i][j]);
        }
    }

    printf("original Matrix:\n");
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            printf("%.2f \t", mat[i][j]);
        }
        printf("\n");
    }
```

```c
    printf("transpose Matrix:\n");
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            trans[i][j] = mat[j][i];
            printf("%.2f \t", trans[i][j]);
        }
        printf("\n");
    }

    float det = mat[0][0] * (mat[1][1] * mat[2][2] - mat[1][2] * mat[2][1]) -
                mat[0][1] * (mat[1][0] * mat[2][2] - mat[1][2] * mat[2][0]) +
                mat[0][2] * (mat[1][0] * mat[2][1] - mat[1][1] * mat[2][0]);

    printf("\nDeterminant: %.2f\n", det);

    cof[0][0] =  (mat[1][1] * mat[2][2] - mat[1][2] * mat[2][1]);
    cof[0][1] = -(mat[1][0] * mat[2][2] - mat[1][2] * mat[2][0]);
    cof[0][2] =  (mat[1][0] * mat[2][1] - mat[1][1] * mat[2][0]);
    cof[1][0] = -(mat[0][1] * mat[2][2] - mat[0][2] * mat[2][1]);
    cof[1][1] =  (mat[0][0] * mat[2][2] - mat[0][2] * mat[2][0]);
    cof[1][2] = -(mat[0][0] * mat[2][1] - mat[0][1] * mat[2][0]);
    cof[2][0] =  (mat[0][1] * mat[1][2] - mat[0][2] * mat[1][1]);
    cof[2][1] = -(mat[0][0] * mat[1][2] - mat[0][2] * mat[1][0]);
    cof[2][2] =  (mat[0][0] * mat[1][1] - mat[0][1] * mat[1][0]);
```

```c

    printf("\nCofactor Matrix:\n");
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            printf("%.2f \t", cof[i][j]);
        }
        printf("\n");
```

```
54
55      printf("\nAdjoint Matrix:\n");
56      for(i = 0; i < 3; i++) {
57          for(j = 0; j < 3; j++) {
58              adj[i][j] = cof[j][i];
59              printf("%.2f \t", adj[i][j]);
60          }
61          printf("\n");
62      }
63
64      if(det == 0) {
65          printf("Inverse is not possible because determinant is 0.\n");
66      } else {
67          printf("Inverse Matrix:\n");
68          for(i = 0; i < 3; i++) {
69              for(j = 0; j < 3; j++) {
70                  inv[i][j] = adj[i][j] / det;
71                  printf("%.2f \t", inv[i][j]);
72              }
73              printf("\n");
74          }
75      }
76
77  }
```

OUTPUT:

```
C:\Users\lenovo\Desktop\Untitled4.exe
enter elements of 3x3 matrix:
1 2 3
0 1 4
5 6 0
original Matrix:
1.00    2.00    3.00
0.00    1.00    4.00
5.00    6.00    0.00
transpose Matrix:
1.00    0.00    5.00
2.00    1.00    6.00
3.00    4.00    0.00

Determinant: 1.00

Cofactor Matrix:
-24.00  20.00   -5.00
18.00   -15.00  4.00
5.00    -4.00   1.00

Adjoint Matrix:
-24.00  18.00   5.00
20.00   -15.00  -4.00
-5.00   4.00    1.00
Inverse Matrix:
-24.00  18.00   5.00
20.00   -15.00  -4.00
-5.00   4.00    1.00

--------------------------------
```

Q#02:

[*] comb.c | Untitled2.cpp | combination.c | [*] Untitled4.cpp | [*] Untitled5

```c
#include <stdio.h>

int main() {
    int rows, cols, i, j, k;
    int mat[5][5];

    printf("Enter number of rows (up to 5): ");
    scanf("%d", &rows);
    printf("Enter number of columns (up to 5): ");
    scanf("%d", &cols);

    printf("Enter matrix elements:\n");
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            scanf("%d", &mat[i][j]);
        }
    }

    printf("\n--- Matrix Types Detected ---\n");

    if(rows == cols) {
        printf("- Square Matrix\n");
    } else {
        printf("- Rectangular Matrix\n");
    }
```

```c
    if(rows == 1) {
        printf("- Row Matrix\n");
    }

    if(cols == 1) {
        printf("- Column Matrix\n");
    }

    int isZero = 1;
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            if(mat[i][j] != 0) {
                isZero = 0;
            }
        }
    }
```

```
43
44      if(isZero) {
45          printf("- Zero Matrix / Null Matrix\n");
46      }
47
48      if(rows == cols) {
49          int isDiagonal = 1;
50          for(i = 0; i < rows; i++) {
51              for(j = 0; j < cols; j++) {
52                  if(i != j && mat[i][j] != 0) {
53                      isDiagonal = 0;
54                  }
55              }
56          }
57
58          if(isDiagonal) {
59              printf("- Diagonal Matrix\n");
60
61              int isScalar = 1;
62              for(i = 0; i < rows; i++) {
63                  if(mat[i][i] != mat[0][0]) {
64                      isScalar = 0;
65                  }
66              }
67              if(isScalar) {
68                  printf("- Scalar Matrix\n");
69              }
70
71              if(isScalar && mat[0][0] == 1) {
72                  printf("- Identity Matrix\n");
73              }
74          }
75
76          int isUpper = 1;
77          for(i = 1; i < rows; i++) {
78              for(j = 0; j < i; j++) {
79                  if(mat[i][j] != 0) {
80                      isUpper = 0;
81                  }
82              }
83          }
84          if(isUpper) {
85              printf("- Upper Triangular Matrix\n");
86          }
87
88          int isLower = 1;
89          for(i = 0; i < rows; i++) {
90              for(j = i + 1; j < cols; j++) {
```

```
            if(mat[i][j] != 0) {
                isLower = 0;
            }
        }
    }
    if(isLower) {
        printf("- Lower Triangular Matrix\n");
    }

    int isSymmetric = 1;
    int isSkewSymmetric = 1;
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            if(mat[i][j] != mat[j][i]) {
                isSymmetric = 0;
            }
            if(mat[i][j] != -mat[j][i]) {
                isSkewSymmetric = 0;
            }
        }
    }

    if(isSymmetric) {
        printf("- Symmetric Matrix\n");
    }
    if(isSkewSymmetric) {
        printf("- Skew-Symmetric Matrix\n");
    }

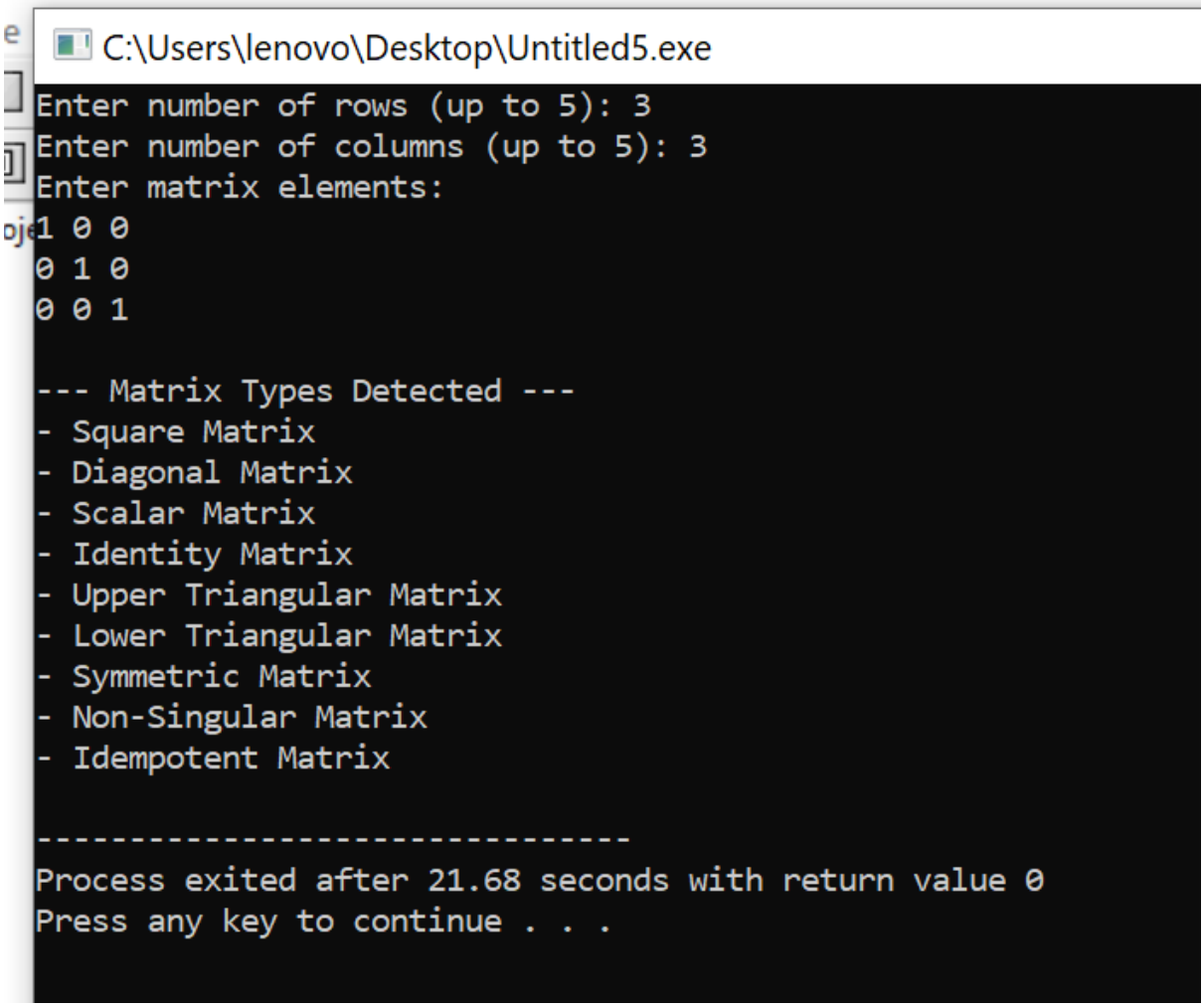
    int det = 0;
    if(rows == 2) {
        det = mat[0][0] * mat[1][1] - mat[0][1] * mat[1][0];
    } else if(rows == 3) {
        det = mat[0][0] * (mat[1][1] * mat[2][2] - mat[1][2] * mat[2][1]) -
              mat[0][1] * (mat[1][0] * mat[2][2] - mat[1][2] * mat[2][0]) +
              mat[0][2] * (mat[1][0] * mat[2][1] - mat[1][1] * mat[2][0]);
    }

    if(rows == 2 || rows == 3) {
        if(det == 0) {
            printf("- Singular Matrix\n");
        } else {
            printf("- Non-Singular Matrix\n");
        }
    }

    int sq[5][5] = {0};
    for(i = 0; i < rows; i++) {
```

```
138         for(i = 0; i < rows; i++) {
139             for(j = 0; j < cols; j++) {
140                 for(k = 0; k < rows; k++) {
141                     sq[i][j] += mat[i][k] * mat[k][j];
142                 }
143             }
144         }
145 
146         int isIdempotent = 1;
147         int isNilpotent = 1;
148         for(i = 0; i < rows; i++) {
149             for(j = 0; j < cols; j++) {
150                 if(sq[i][j] != mat[i][j]) {
151                     isIdempotent = 0;
152                 }
153                 if(sq[i][j] != 0) {
154                     isNilpotent = 0;
155                 }
156             }
157         }
158 
159         if(isIdempotent) {
160             printf("- Idempotent Matrix\n");
161         }
161         }
162         if(isNilpotent) {
163             printf("- Nilpotent Matrix\n");
164         }
165     }
166 
167     return 0;
168 }
```

OUTPUT


C:\Users\lenovo\Desktop\Untitled5.exe
Enter number of rows (up to 5): 3
Enter number of columns (up to 5): 3
Enter matrix elements:
1 0 0
0 1 0
0 0 1
--- Matrix Types Detected ---
- Square Matrix
- Diagonal Matrix
- Scalar Matrix
- Identity Matrix
- Upper Triangular Matrix
- Lower Triangular Matrix
- Symmetric Matrix
- Non-Singular Matrix
- Idempotent Matrix

Process exited after 21.68 seconds with return value 0
Press any key to continue . . .

Q#03:

[*] comb.c | Untitled2.cpp | combination.c | [*] Untitled4.cpp | [*] Untitled5.cpp

```c
#include <stdio.h>

int main() {
    int r1, c1, r2, c2, i, j, k;
    int A[3][3], B[3][3], C[3][3];

    printf("Enter rows and columns for Matrix A (max 3x3): ");
    scanf("%d %d", &r1, &c1);

    printf("Enter rows and columns for Matrix B (max 3x3): ");
    scanf("%d %d", &r2, &c2);

    if (c1 != r1 && c1 != r2) {
        printf("\nError: Matrix multiplication not possible. Columns of A must equal rows of B.\n");
        return 0;
    }

    if (c1 != r2) {
        printf("\nError: Matrix multiplication not possible. Columns of A (%d) must equal rows of B (%d).\n", c1, r2);
        return 0;
    }

    printf("\nEnter elements of Matrix A:\n");
    for (i = 0; i < r1; i++) {
```

[*] comb.c | Untitled2.cpp | combination.c | [*] Untitled4.cpp | [*] Untitled5.cpp

```c
            scanf("%d", &A[i][j]);
        }
    }

    printf("\nEnter elements of Matrix B:\n");
    for (i = 0; i < r2; i++) {
        for (j = 0; j < c2; j++) {
            scanf("%d", &B[i][j]);
        }
    }

    for (i = 0; i < r1; i++) {
        for (j = 0; j < c2; j++) {
            C[i][j] = 0;
            for (k = 0; k < c1; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    printf("\nMatrix A:\n");
    for (i = 0; i < r1; i++) {
        for (j = 0; j < c1; j++) {
            printf("%d\t", A[i][j]);
```

```
50          }
51          printf("\n");
52      }
53
54      printf("\nMatrix B:\n");
55      for (i = 0; i < r2; i++) {
56          for (j = 0; j < c2; j++) {
57              printf("%d\t", B[i][j]);
58          }
59          printf("\n");
60      }
61
62      printf("\nResultant Matrix (A x B):\n");
63      for (i = 0; i < r1; i++) {
64          for (j = 0; j < c2; j++) {
65              printf("%d\t", C[i][j]);
66          }
67          printf("\n");
68      }
69
70      return 0;
71  }
```

```
C:\Users\lenovo\Desktop\Untitled5.exe
Enter rows and columns for Matrix A (max 3x3): 2 3
Enter rows and columns for Matrix B (max 3x3): 3 2

Enter elements of Matrix A:
1 2 3
4 5 6

Enter elements of Matrix B:
7 8
9 1
2 3

Matrix A:
1       2       3
4       5       6

Matrix B:
7       8
9       1
2       3

Resultant Matrix (A x B):
31      19
85      55

--------------------------------
Process exited after 32.01 seconds with return value 0
Press any key to continue . . .
```

Q#04:

```c
#include <stdio.h>

int main() {
    int arr[3][3][3];
    int k, i, j;
    int grandTotal = 0;
    int minVal, maxVal;

    printf("Enter elements for 3x3x3 array (27 numbers):\n");
    for (k = 0; k < 3; k++) {
        for (i = 0; i < 3; i++) {
            for (j = 0; j < 3; j++) {
                scanf("%d", &arr[k][i][j]);
            }
        }
    }

    minVal = arr[0][0][0];
    maxVal = arr[0][0][0];

    printf("\n--- Layer-Wise Analysis ---\n");
    for (k = 0; k < 3; k++) {
        int layerSum = 0;
        printf("\nLayer %d:\n", k + 1);
```

```c
    minVal = arr[0][0][0];
    maxVal = arr[0][0][0];

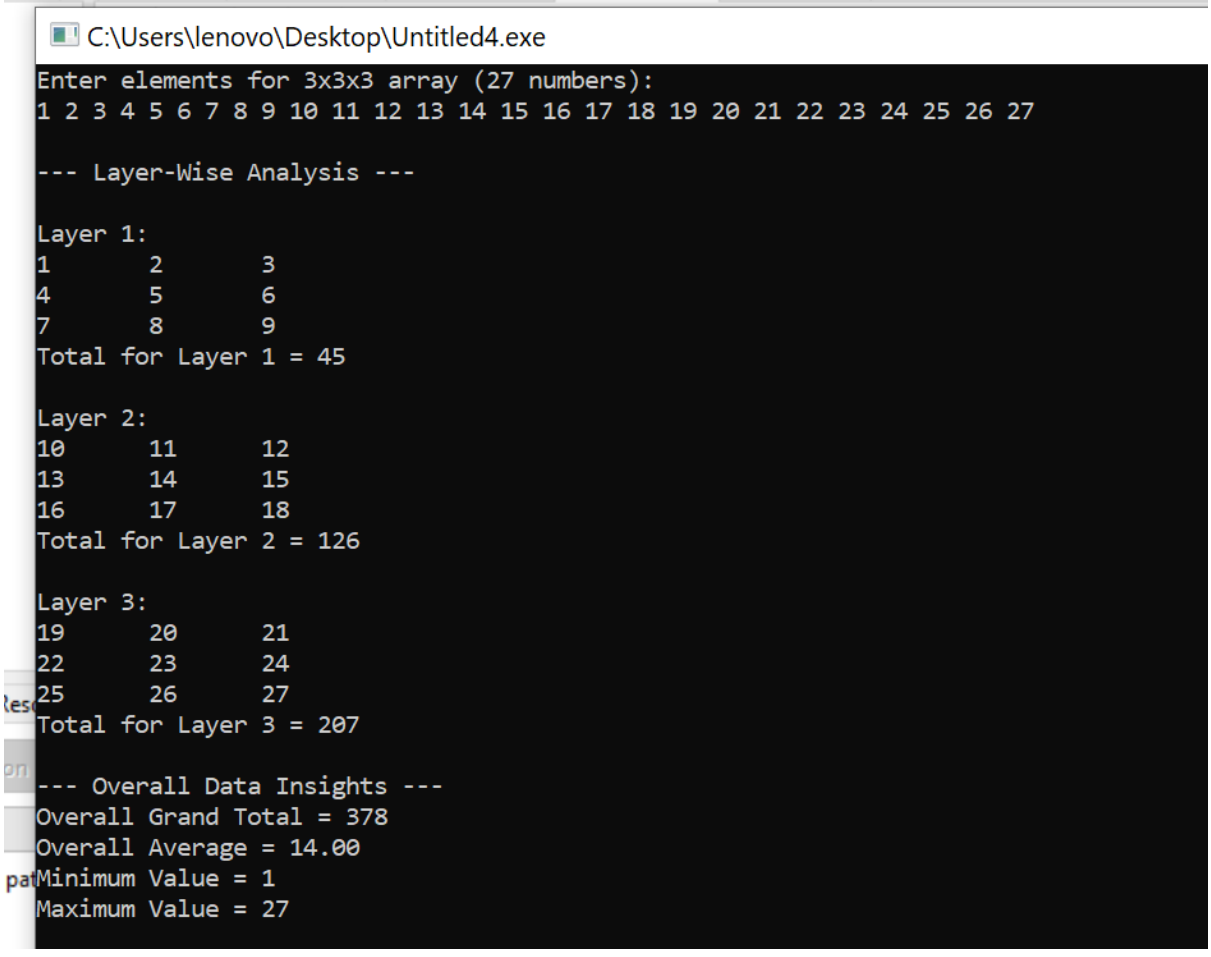
    printf("\n--- Layer-Wise Analysis ---\n");
    for (k = 0; k < 3; k++) {
        int layerSum = 0;
        printf("\nLayer %d:\n", k + 1);
        for (i = 0; i < 3; i++) {
            for (j = 0; j < 3; j++) {
                printf("%d\t", arr[k][i][j]);
                layerSum += arr[k][i][j];
                grandTotal += arr[k][i][j];

                if (arr[k][i][j] < minVal) {
                    minVal = arr[k][i][j];
                }
                if (arr[k][i][j] > maxVal) {
                    maxVal = arr[k][i][j];
                }
            }
            printf("\n");
        }
        printf("Total for Layer %d = %d\n", k + 1, layerSum);
    }
```



```
[*] comb.c | Untitled2.cpp | combination.c | [*] Untitled4.cpp | Untitled5.cpp

27                  printf("%d\t", arr[k][i][j]);
28                  layerSum += arr[k][i][j];
29                  grandTotal += arr[k][i][j];
30
31                  if (arr[k][i][j] < minVal) {
32                      minVal = arr[k][i][j];
33                  }
34                  if (arr[k][i][j] > maxVal) {
35                      maxVal = arr[k][i][j];
36                  }
37              }
38              printf("\n");
39          }
40          printf("Total for Layer %d = %d\n", k + 1, layerSum);
41      }
42
43      printf("\n--- Overall Data Insights ---\n");
44      printf("Overall Grand Total = %d\n", grandTotal);
45      printf("Overall Average = %.2f\n", grandTotal / 27.0);
46      printf("Minimum Value = %d\n", minVal);
47      printf("Maximum Value = %d\n", maxVal);
48
49      return 0;
50  }
```



```
C:\Users\lenovo\Desktop\Untitled4.exe
Enter elements for 3x3x3 array (27 numbers):
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27

--- Layer-Wise Analysis ---

Layer 1:
1       2       3
4       5       6
7       8       9
Total for Layer 1 = 45

Layer 2:
10      11      12
13      14      15
16      17      18
Total for Layer 2 = 126

Layer 3:
19      20      21
22      23      24
25      26      27
Total for Layer 3 = 207

--- Overall Data Insights ---
Overall Grand Total = 378
Overall Average = 14.00
Minimum Value = 1
Maximum Value = 27
```

Q#05

[*] comb.c | Untitled2.cpp | combination.c | [*] Untitled4.cpp | Untitled5.cpp

```c
#include <stdio.h>

int main() {
    int arr[3][3][3];
    int k, i, j;

    printf("Enter elements for 3x3x3 array (27 numbers):\n");
    for (k = 0; k < 3; k++) {
        for (i = 0; i < 3; i++) {
            for (j = 0; j < 3; j++) {
                scanf("%d", &arr[k][i][j]);
            }
        }
    }

    printf("\n--- Layers Overview ---\n");
    for (k = 0; k < 3; k++) {
        printf("\nLayer %d:\n", k + 1);
        for (i = 0; i < 3; i++) {
            for (j = 0; j < 3; j++) {
                printf("%d\t", arr[k][i][j]);
            }
            printf("\n");
        }
    }
```

urces | Compile Log | Debug | Find Results | Close

Compilation results...

[*] comb.c | Untitled2.cpp | combination.c | [*] Untitled4.cpp | Untitled5.cpp

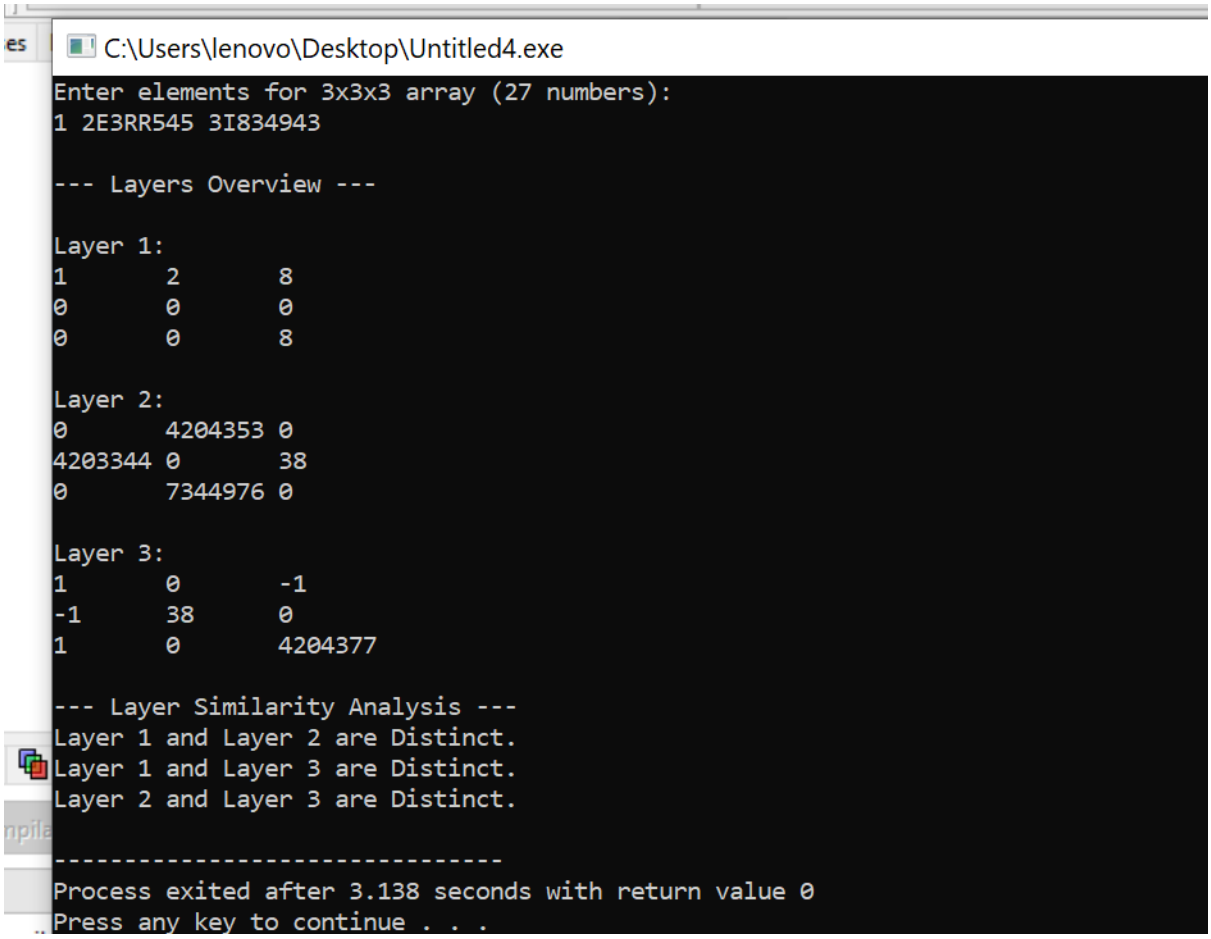
```c
    }
    if (same13) {
        printf("Layer 1 and Layer 3 are Identical.\n");
    } else {
        printf("Layer 1 and Layer 3 are Distinct.\n");
    }

    int same23 = 1;
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            if (arr[1][i][j] != arr[2][i][j]) {
                same23 = 0;
            }
        }
    }
    if (same23) {
        printf("Layer 2 and Layer 3 are Identical.\n");
    } else {
        printf("Layer 2 and Layer 3 are Distinct.\n");
    }

    return 0;
}
```

ces | Compile Log | Debug | Find Results | Close

Compilation results...


C:\Users\lenovo\Desktop\Untitled4.exe
Enter elements for 3x3x3 array (27 numbers):
1 2E3RR545 3I834943
--- Layers Overview ---
Layer 1:
1 2 8
0 0 0
0 0 8
Layer 2:
0 4204353 0
4203344 0 38
0 7344976 0
Layer 3:
1 0 -1
-1 38 0
1 0 4204377
--- Layer Similarity Analysis ---
Layer 1 and Layer 2 are Distinct.
Layer 1 and Layer 3 are Distinct.
Layer 2 and Layer 3 are Distinct.

Process exited after 3.138 seconds with return value 0
Press any key to continue . . .